

TP3 - Getting data from the web

Mobile & Ubiquitous Computing 2026/2027

Ciências, Departamento de Informática

Diogo Branco

More from last week

Sealed classes and interfaces

- restricted class hierarchies
- control over inheritance
- subclasses of a sealed class are known at compile time

Adapt to different Screen Sizes

<https://developer.android.com/courses/pathways/android-basics-compose-unit-4-pathway-3>

Getting data from the web

#3 Class

Evaluation

Assignment #2 - Dice Screens

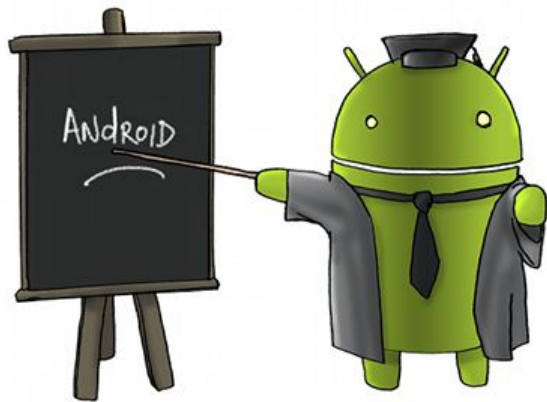
Goals

App Architecture

Retrieve data from the web

Display Data

01



App Architecture

Why?

- Mobile devices are also resource-constrained
 - Your app can be killed at any time
- Your app is not sequential
 - Screens can be opened and closed out of order

Thus, Data should not be stored in app components (e.g. activities)

Roles to play - *Modern App Architecture

UI Layer (e.g. Activities, fragments, composables)

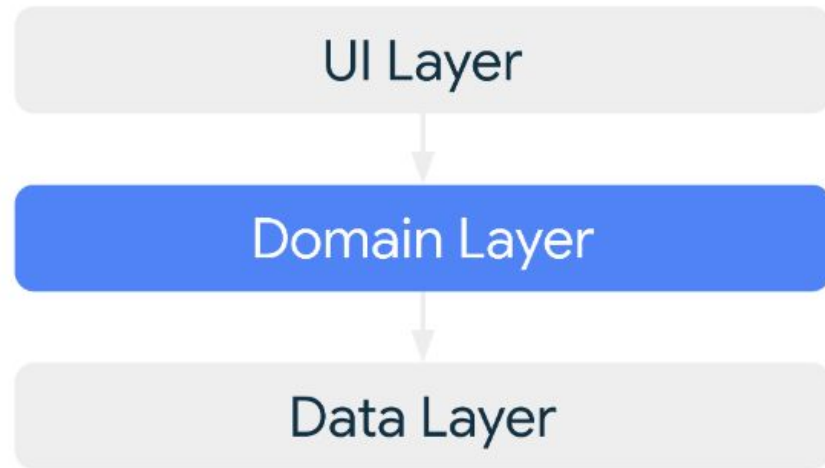
- UI logic only

Domain Layer (Optional)

- Reusable logic for different viewmodels

Data Layer (e.g. Data Models)

- Persistence models of data
 - Used to populate UI
 - Affected by user interactions
- Single source of truth
- Business Logic



Learn about architecture

<https://developer.android.com/topic/architecture>

UI Layer

- **UI elements**
- **ViewModels**
 - hold the data
 - expose data to the UI
 - handle the app logic
 - ViewModel objects are not destroyed
 - Available for recomposition



Flow

- The `ViewModel` holds and exposes the state the UI consumes.
- The UI state is application data transformed by the `ViewModel`.
- The UI notifies the `ViewModel` of user events.
- The `ViewModel` handles the user actions and updates the state.
- The updated state is fed back to the UI to render.
- This process repeats for any event that causes a mutation of state.

02



MarsPhoto

**Fetch data from the
internet**

Tutorial - Get data from the internet

Mars Photos

Download the zip, or make sure you go to the starting branch in git

(In Android Studio VCS -> Get From Version Control)

1. Follow the tutorial (Get data from the internet

8. Step 8 needs next slide!

<https://tinyurl.com/CMInternet>



Tutorial - Get data from the internet

Fixes

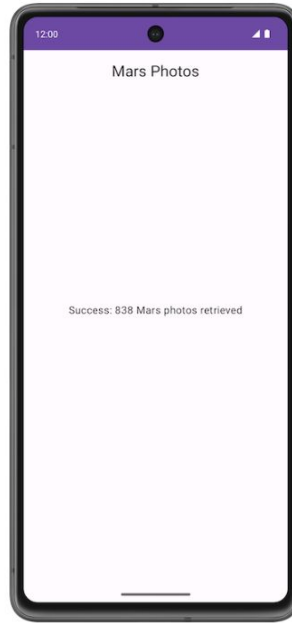
Always use the most updated versions

If Error in Serializable:

Add: `import kotlinx.serialization.Serializable` (to `MarsPhoto.kt`)

You should have a working app

Run!



Display data

Tutorial - Display

Mars Photos

Download the zip, or make sure you go to the **repo-starter branch in git**

(In Android Studio Git -> Pull -> Starter)

00. It will have additional code. Inspect the differences!

Tutorial - Displaying Images

2. Add to build.gradle (and sync)

```
// Coil  
implementation("io.coil-kt:coil-compose:2.4.0")
```

3. On MarsViewModel.kt sealed interface

```
sealed interface MarsUiState {  
    data class Success(val photos: String, val randomPhoto : MarsPhoto) : MarsUiState  
    object Error : MarsUiState  
    object Loading : MarsUiState  
}
```

4. On MarsViewModel.kt getMarsPhotos, add to success a random image retrieved

```
MarsUiState.Success( "Success: ${listResult.size} Mars photos retrieved", listResult.random())
```

Tutorial - Displaying Images

5. To display on the screen, in HomeScreen.kt

```
is MarsUiState.Success -> ResultScreen(  
    marsUiState.photos, marsUiState.randomPhoto, modifier = modifier.fillMaxWidth()  
)
```

6. Display image using AsyncImage from coil (in HomeScreen.kt)

```
import coil.compose.AsyncImage  
import coil.request.ImageRequest  
import com.example.marsphotos.network.MarsPhoto
```

@Composable

```
fun ResultScreen(photos: String, randomPhoto: MarsPhoto, modifier: Modifier = Modifier) {  
    Column(  
        modifier = modifier  
    ) {  
        Text(text = photos)  
  
        AsyncImage(  
            model = ImageRequest.Builder(LocalContext.current)  
                .data(randomPhoto.imgSrc)  
                .crossfade(true)  
                .build(),  
            contentDescription = "A photo", )    }}
```

It should look like this



Learn more about coil

<https://coil-kt.github.io/coil/compose/>

Other Useful Resources

Recommended

Tutorial about App Architecture

Download the zip, or make sure you go to the starting branch in git

<https://tinyurl.com/CMArch>

Additional resources

Coroutines Learn More

<https://developer.android.com/topic/libraries/architecture/coroutines>

Coroutines Playground

<https://tinyurl.com/CMCoroutines>

Coroutines Android

<https://tinyurl.com/CMCoroutinesAndroid>

Learn more about Retrofit

<https://github.com/lysine-dev/retrofit>

Assignment #3 - Rick and Morty Explorer

Create a new feature from scratch that connects to the public Rick and Morty API (rickandmortyapi.com).

The application must show how many images you are retrieving from the API, and randomly choose one to display. Every time the app opens a random image should be displayed. The item to display must include its name, the image, and two other relevant fields from its specific JSON schema.

Selection component (e.g., a Dropdown/Spinner or Selection Buttons) allowing the user to choose between three search categories: Character, Episode, or Location.

Filtering inside the selected category, the user can filter the results by entering a name into a text input field.

Roll button that randomly updates to another item

Tip - You will need to construct the BASE_URL properly, design different data models (Data Classes) to map each specific JSON structure, and configure query parameters dynamically based on the user's selection.

